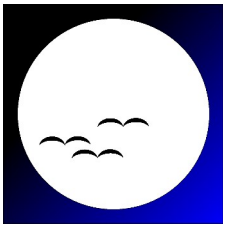


QSpice C-Block Components

Programmable Attributes – Lookup Symbols

Revision Date: 2026.03.29

Caveats & Cautions

First and foremost: Do not assume that anything I say is true until you test it yourself. In particular, I have no insider information about QSpice. I've not seen the source code, don't pretend to understand all of the intricacies of the simulator, or, well, anything. Trust me at your own risk.

If I'm wrong, please let me know. If you think this is valuable information, let me know. (As Dave Plummer, ex-Microsoft developer and YouTube celebrity (Dave's Garage) says, "I'm just in this for the likes and subs.")

Note: This is the tenth document in a clearly unplanned series:

- ***The first C-Block Basics document covered how the QSpice DLL interface works.***
- ***The second C-Block Basics document demonstrated techniques to manage multiple schematic component instances, multi-step simulations, and shared resources.***
- ***The third C-Block Basics document built on the second to demonstrate a technique to execute component code at the beginning and end of simulations.***
- ***The fourth C-Block Basics document revisited the Trunc() function and provided a basic "canonical" use example.***
- ***The fifth C-Block Basics document revisited the MaxExtStepSize() function with a focus on generating reliable clock signals from within the component.***
- ***The sixth C-Block Basics document examined connecting and using QSpice bus wires with components.***
- ***The seventh C-Block Basics document described recent QSpice changes (a new Display() function changes to Trunc()).***
- ***The eighth C-Block Basics document detailed post-processing techniques and much more in painful detail.***
- ***The ninth C-Block Basics document explained how to use the QSpice EngAtof() function.***
- ***The tenth C-Block Basics document was a technical reference for the QSpice "Berkeley Sockets API."***
- ***The eleventh C-Block Basics document extended the "Berkeley Sockets API" framework to overcome the limitations of template-generated code.***
- ***The twelfth C-Block Basics document created multi-client "Berkeley Sockets" host servers for C++, Python, and Java.***
- ***The thirteenth C-Blocks Basics document demonstrated a technique to exchange data directly between multiple DLLs and why you probably shouldn't do it.***
- ***The fourteenth C-Block Basics document provided techniques for creating C-Block component symbols and basic information for using Programmable Attributes.***

This paper builds on the prior paper with information about using Programmable Attribute Lookup symbols. See my [GitHub repository here] for prior documents in this series.

Overview

In the last edition of this unplanned series, we looked at using QSpice symbols with C-Block components. In Step 4, we learned a bit about Programmable Attributes and how we might use them to provide user-friendly drop-down lists and range-limited attributes in the Symbol Properties pane.

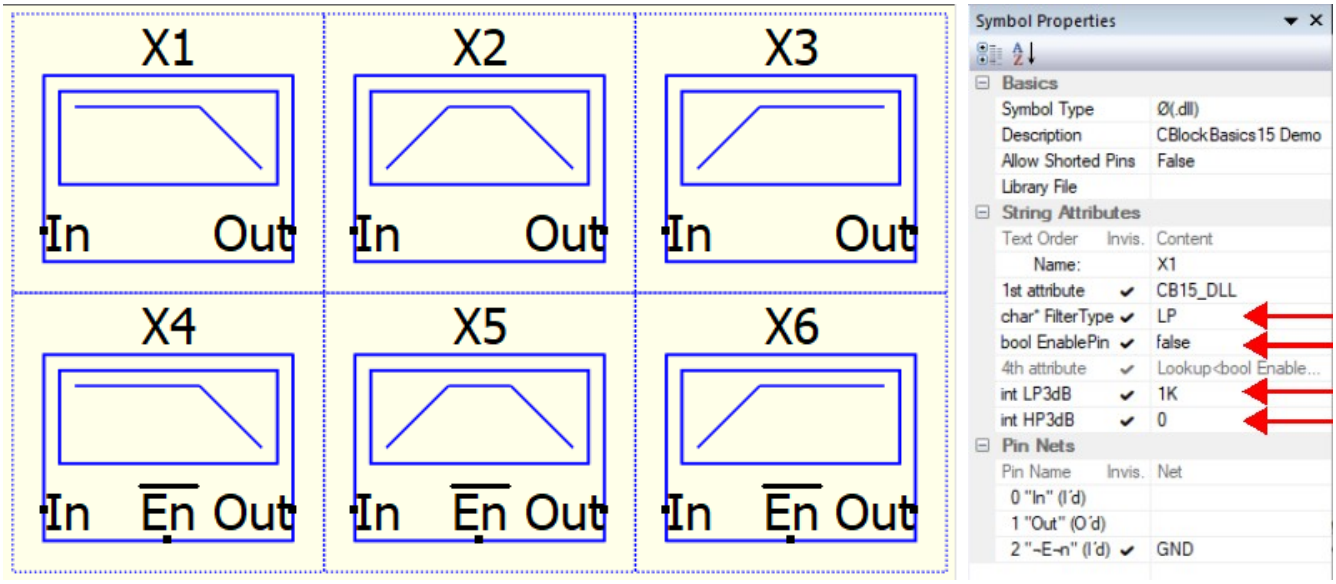
This time we'll investigate the Lookup Programmable Attribute. (See *QSpice Help, Schematic Capture → Symbol Editor → Programmable Attributes*). It's pretty cool. But the documentation is a bit incomplete with respect to DLL component code.

What Can We Do With Lookup?

Before getting into the details, let's get an idea of what can be done with the Lookup Programmable Attribute.

Let's say that we want to create a C-Block component to implement a first-order filter symbol that can be dropped onto a schematic. We want the user to be able to select low-pass, high-pass, and band-pass from a drop-down list in the Symbol Properties and specify 3dB cutoffs. We want to allow the user to optionally include an "enable pin" on the symbol. Finally, we want the component symbol graphics to change to indicate the filter selection visually.

When the user drops the symbol on a schematic, the initial symbol will look like X1 with the Symbol Properties shown on the right. The user can switch between filter types with drop-down menus in the Symbol Properties. He can add/remove the enable pin (EN) in the same way. When the user changes the filter type and enable pin choices, the schematic symbol changes to reflect the choices as in X2-X6 below. Finally, he can edit the applicable 3dB cutoff values as well.



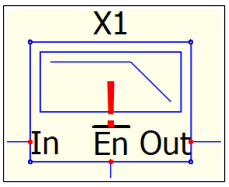
Installing The Demo Files

To "play along," you are going to need administrative rights to the QSpice installation folder. Unfortunately, this is unavoidable at this time[1].

Navigate to the QSpice installation folder, typically "`C:\Program Files\QSpice`." There should be a subfolder named Repository and it should be empty. Create a new subfolder named CB15_Filter (i.e., "`C:\Program Files\QSpice\Repository\CB15_Filter`"). Copy the following demo files into that folder:

- CB15_LP.qsym
- CB15_HP.qsym
- CB15_BP.qsym
- CB15_LPEN.qsym
- CB15_HPEN.qsym
- CB15_BPEN.qsym

The CB15_Filter.qsym, CB15_Demo.qsch, and CB15_DLL.cpp files can be placed anywhere but I suggest that you put them together in a new, empty folder for now. Compile the *.cpp file.



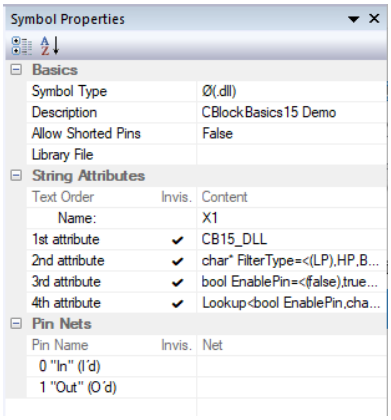
You should be able to open CB15_Demo.qsch and change the filter type, enable pin, and 3dB settings. If the CB15_Filter symbol displays a red exclamation point when you hover the cursor over the symbol, something went wrong. There should be a message in the bottom message area of the QSpice window. Be sure that you didn't rename any of the files, named the Repository subfolder `CB15_Filter`, and copied the *.qsym files there as described above.

Assuming no issues so far, you can create a new schematic and drag the CB15_Filter.qsym symbol from File Explorer onto the schematic. (Add the folder containing CB15_Filter.qsym to the Symbols & IP Browser if you wish to drag from there.) Play with the Symbol Browser settings to get a feel for the way the Lookup Attribute works. Examine the demo files, make changes, break things.

With that done, let's get into how this Lookup Programmable Attribute thing works….

The Top-Level Symbol

Open CB15_Filter.qsym. The symbol properties are what you'd expect for a DLL component symbol – Symbol Type = Ø(.dll), Name = X1, 1st String Attribute = DLL name. There are two ports (In and Out).



There are two Multiple-Choice Programmable Attributes (MCPAs):

```
char* FilterType=<(LP),HP,BP>
bool EnablePin=<(false),true>
```

These define the drop-down lists for the Symbol Properties.

Now let's make a distinction between MCPA *values* and *indexes*. The *value* of a MCPA is the list item chosen by the user in Symbol Properties.

1 I've asked Mike Engelhardt, author of QSpice, to make the "Repository Folder" a user-configurable location but, for now, it's hard-coded.

The *index* of a MCPA is the zero-based position of the selected item in the MCPA list. The index will be important for Lookup Attributes.

Note that our FilterType and EnablePin MCPAs have data types. This is needed to pass those *values* to the DLL. For example, if the user selects "LP", the DLL will receive "LP" as a char* in the uData FilterType element. (Without the data type declarations, they would not be passed into a DLL. They would be available to, say, a library file model for a non-DLL symbol.)

Then there's the Lookup Programmable Attribute:

```
Lookup<bool EnablePin,char* FilterType;CB15_LP,CB15_HP,CB15_BP,CB15_LPEN,CB15_HPEN,CB15_BPEN>
```

Because we gave data types to EnablePin and FilterType, we must include those types in the Lookup attribute. Without the type declarations in Lookup, QSpice will complain that it cannot find EnablePin and FilterType. (Again, omit the data types if you don't need them in a DLL symbol.)

So, the Lookup attribute has two indexes. (You might need only one or you might need more. I'm demonstrating a 2-D version.) With FilterType index = [0-2] and EnablePin index [0-1], there are six possible outcomes. These are mapped to the six symbol file names (omitting the `.qsym` filename extension) in the `[QSpiceIntallDir]\Repository\CB15_Filter` folder. (I assume that the indexing order is clear.)

Note: In case it's not obvious, the second-level symbol file names need not be unique. The could, in fact, all be the same, i.e., point to a single second-level symbol. That might be useful during development.

Note: When you add a symbol to a schematic, the symbol file is copied into the schematic. If you modify the symbol file, you must delete/re-add the symbol to the schematic for the changes to take effect.

Note: The top-level MCPAs in my example all contain valid default values (i.e., the value in parenthesis). If you don't specify a default value, well, just supply one to simplify your life….

OK, that's how Lookup maps to the second-level `*.qsym` files. Next, we'll investigate how the second-level files are merged into the top-level symbol.

Second-Level Symbol Files

Looking at the top-level symbol (`CB15_Filter.qsym`), you might notice that there is not an Enable pin (EN). The 3dB cutoff values are missing. The filter graphic is missing. But all of them are present in the schematic.

The missing items are present in the second-level symbols (`CB15_LP.qsym`, `CB15_LPEN.qsym`, etc.). In fact, the "non-EN" versions have Enable pins even though they are not used. (They are tied to GND and, therefore, aren't displayed in the schematic although still present in the Symbol Browser.)

The 3dB values are defined with ranges. We can't prevent the user from editing them (AFAIK) but we set the range to be 0:0 for LP/HP selections so the user will realize when they don't apply.

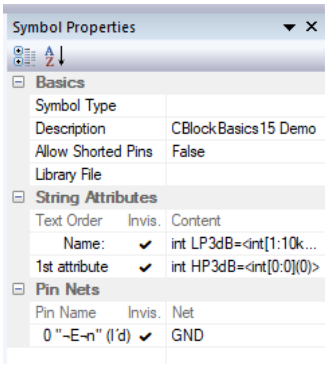
When the user changes Lookup selections, the merge process works like this:

- Any previously merged second-level elements are removed.
- The new second-level symbol is copied into the top-level symbol inside the schematic.
- The second-level pins and attributes are copied *in the order that they are defined* in the second-level symbol.

That last item is likely to be a source of problems. The order of pins and attributes in a symbol depends on the order that they were added. You can right-click pins in the symbol editor to change the order after they are added. But attributes don't have this convenient feature – the order in which you add them is the order they will have in the symbol. (You can reorder attributes with judicious use of copy/paste/delete.)

This is key: *The order, data types, and data direction must be the identical for all second-level symbol pins and attributes because this determines the order and types of data passed to the DLL in uData.*

So, my recommendation for creating second-level symbols is this: Set the first one up with all needed pins and attributes. Once you're satisfied, make copies for the remaining second-level symbols. Then edit these carefully – do not add/delete anything, just edit and move. Don't modify pin types/direction or attribute data types. Once that's done, test all of the user-selections in a schematic paying attention to the Symbol Browser – if elements move up/down in the browser, then something's wrong. You can also check the netlists generated with each combination of selections.



A final note before moving on: The Symbol Type of second-level symbols is, as far as I can tell, not used. Also, the Name string attribute isn't really a symbol name, it's just the first attribute. See example from `CB15_LP.qsym` to the right.

OK, enough of that. On to how to create the DLL.

Creating The DLL Component

Once the symbols are completed, DLL creation is pretty much the same as for any symbol. Add the top-level symbol to a schematic and right-click to generate a template. See C-Block Basics #14 for more information. The demonstration code for this project may also help (`CB15_DLL.cpp`).

Remapping The Repository Folder

Adding/deleting files and folders in the default QSpice repository location typically requires Administrator rights. You'll be prompted each time you make changes. It's annoying and inconvenient.

You can delete/rename the default Repository folder and replace it with a symbolic link (a Junction Point) to another folder. To do this manually, see this Microsoft Help page (mklink command).

As an alternative, I've created – or, to be honest, Claude.AI created – a batch file to do this (CB15_`make_junction.bat`). *You'll need to edit the batch file to set the location of your desired custom Repository folder.* See the instructions in the batch file.

When run, the batch file will rename the existing QSpice Repository folder (in case it's not actually empty) and create a Junction Point link named Repository to your custom repository folder. If not run as Administrator, you will be prompted to continue. QSpice updates don't remove the link. However, uninstalling will delete the link. If you re-install QSpice, you can run the batch file again to restore the link.

Acknowledgments

My thanks to QSpice forum users @LPoma and @KSKelvin for their help working out the Lookup Attribute and for their ongoing contributions to the QSpice community. Any errors or misinformation in this document is, of course, solely my fault. Thanks, guys!

Conclusion

I hope that you find the information useful. And, of course, let me know if you find problems or suggestions for improving the code or this documentation.

--robert